



Patuakhali Science and Technology University
Faculty of Computer Science and Engineering

Software Development Project – I
Project Report

Smart Due Manager

Customer Due & Payment Management with Automated SMS Reminders

Project Title : Smart Due Manager (Customer Due and Payment Management System)
Submission Date : 04, August 2026

Submitted from, Dipjoy Bal ID : 2102053 Reg : 10180 Department of Computer Science and Information Technology	Submitted to, 1. Dr. Md. Samsuzzaman Professor, Department of Computer and Communication Engineering, Patuakhali Science and Technology University. 2. Sarna Majumder Associate Professor, Department of Computer and Communication Engineering, Patuakhali Science and Technology University.
---	---

Contents

1. Introduction	3
2. Objectives	3
3. Problem Statement	3
4. Related Work	3
5. Scope	4
6. Key Features	4
7. Methodology	4
7.1. Step-by-Step Development Methodology	5
8. Visual Models	5
8.1. Flow Chart Diagram	5
8.2. ERD (Entity Relationship Diagram)	7
8.3. Timeline (Gantt Chart)	7
8.4. UI/UX Design	8
8.5. App screens	8
9. Limitations	8
10. Future Plans	8
11. Result	9
References	9

Smart Due Manager

1. Introduction

Across small towns and neighborhoods in Bangladesh, a large share of retail commerce — grocery (“মুদি”) shops, pharmacies, and other local vendors — runs partly on informal credit, with customer dues tracked in a handwritten ledger (“বাকির খাতা”). This manual approach is slow to search, offers no automated reminders, and provides no backup if the ledger is lost or damaged. To address this, **Smart Due Manager** is proposed — a cross-platform mobile application, built with Flutter and Firebase, that digitizes customer due tracking and payment recording and, uniquely, reminds customers automatically through SMS sent directly from the shop owner’s own phone, without requiring a paid third-party SMS gateway.

2. Objectives

The main objective of this project is to develop a mobile-based system that allows:

1. Easy and secure recording of customers, payments, and outstanding dues
2. Automated, schedulable SMS reminders sent directly from the owner’s own device
3. Transparent, auditable payment history with PDF reports and customer statements
4. Secure, isolated multi-user data storage, protected by an app-lock and database security rules
5. A fully Bengali-localized experience with a customizable SMS reminder template

3. Problem Statement

Managing customer credit and due collection in small shops is often a manual, disorganized, and inefficient process. Shop owners typically rely on a paper ledger, memory, or phone calls to track balances and remind customers — which leads to:

1. Slow, error-prone lookup of a customer’s current balance
2. No automated way to remind a customer that a payment is due
3. No backup — a lost or damaged ledger means lost records
4. Difficulty producing a printable/shareable proof of transactions
5. No way to safely share the ledger with staff without risking tampering

4. Related Work

1. **Khatabook** — A widely used South Asian digital “খাতা” (credit-book) app that lets shopkeepers record customer dues digitally.
Limitation: Reminder delivery is generally built around the customer also having the Khatabook app (or a linked messaging integration under its own paid plans), rather than a plain SMS sent directly from the shop owner’s own SIM at no extra cost.^[1]
2. **OkCredit** — A popular digital credit-book and payment-reminder app for small merchants.
Limitation: Primarily oriented around the Indian market and UPI-based payments; less tailored to Bangladeshi currency, bKash/Nagad payment methods, or a fully Bengali, business-branded SMS template.^[2]
3. **Generic accounting / invoicing software** (e.g. spreadsheet-based tools or general invoicing apps) — Can record transactions digitally.
Limitation: Built around an invoice-per-transaction workflow rather than a simple running “due” balance per customer, and with no built-in SMS reminder automation.
4. **Manual Systems (Paper Ledger / Excel / Phone Calls)** — Most small shop owners in Bangladesh still track dues on paper, in a spreadsheet, or by calling customers.

^[1]Khatabook. Accessed: Aug. 04, 2026. Online. Available: <https://khatabook.com/>

^[2]OkCredit. Accessed: Aug. 04, 2026. Online. Available: <https://okcredit.in/>

Limitation: Prone to human error, offers no search or backup, no automated reminders, and no digital proof of transactions.

5. Scope

Smart Due Manager is developed as a cross-platform mobile application using Flutter, with Android as the primary target platform because two of its core features — direct SMS sending and background SMS scheduling — depend on Android-specific telephony and background-execution APIs. The backend (authentication, database, file storage, and crash reporting) is provided by Google Firebase. The project covers customer and payment management, reminders and SMS automation, PDF reporting, CSV backup, settings/theming, and an application-level lock screen. It is designed to be simple, scalable, and localized, making it suitable for individual shop owners and small retail businesses in Bangladesh. Point-of-sale/inventory management is intentionally out of scope.

6. Key Features

1. Customer & Payment Management

Add, edit, and search customer profiles; the outstanding due balance updates automatically with every payment or new due entry, and a full payment history is kept per customer (amount, method — bKash, Nagad, Hand Cash, or Bank — and an optional receipt photo).

2. Automated SMS Reminders

Set a reminder date for a customer and the app sends an SMS directly from the owner's own SIM — either immediately or automatically in the background at the scheduled time — with retry-on-failure logic and a per-customer send log.

3. PDF Reports & Customer Statements

Generate daily, weekly, or monthly collection reports, and a printable/shareable PDF statement for any individual customer.

4. CSV Backup & Restore

Export all customers to a CSV file and share it as a backup; re-import from CSV later, with automatic duplicate detection by phone number.

5. Archive & Restore Customers

Soft-delete (archive) a customer without losing their history, and restore them — or permanently delete a customer and all related records — at any time.

6. App Lock & Biometric Security

Protect the app with a PIN (hashed using PBKDF2-HMAC-SHA256, never stored in plain text) or biometric unlock (fingerprint/face), with an escalating lockout after repeated failed attempts.

7. Multi-User Data Isolation

Every customer record is scoped to its owner and enforced by Firestore Security Rules, so multiple shop owners can safely share the same backend without seeing each other's data.

8. Customizable Settings & Theming

Choose a dark or light theme and an accent color, set the currency symbol, adjust the app-wide font scale, and configure a business profile (name, address, logo) used across reports.

9. Bengali Localization

A fully Bengali-capable interface with a bundled Bengali font, plus a customizable SMS reminder template supporting placeholders such as {name}, {amount}, {due_date}, and {business_name}.

7. Methodology

The development of Smart Due Manager followed an Agile approach using Iterative and Incremental Development, enabling rapid prototyping, continuous feedback, and flexible improvement based on real usage.

7.1. Step-by-Step Development Methodology

1. Requirement Analysis

1. Study the core workflow of a small shop owner tracking customer dues on paper
2. Identify core modules — customer/payment management, reminders, reports, backup, security
3. Finalize functional and non-functional requirements
4. Define user stories for the shop-owner actor

2. System Design

1. Design a layered architecture (presentation, state, service, repository, backend)
2. Define the Cloud Firestore schema for users, customers, payments, reminders, and SMS logs
3. Plan per-user data isolation via an `ownerId` field and Firestore Security Rules

3. Technology Stack Selection

1. Frontend: Flutter (Android primary target; Linux/Windows/Web build targets available)
2. Backend: Firebase — Authentication, Cloud Firestore, Storage, Crashlytics
3. Background execution: Workmanager, to schedule SMS independent of the app's lifecycle
4. SMS: `another_telephony`, for direct on-device SMS sending
5. PDF generation: the `pdf` and printing Dart packages
6. Security: `crypto` (PBKDF2-HMAC-SHA256) for PIN hashing, `local_auth` for biometrics

4. Implementation and Testing

1. Features implemented and validated incrementally, module by module
2. Automated unit tests cover the PIN-hashing service; remaining modules verified through manual functional testing against a live Firebase project

Notes

1. Iterations shipped incremental features (auth → customer/payment → reminders/SMS → reports/backup → security) for early testing
2. Security practices include PBKDF2-hashed PINs, constant-time verification, escalating lockout on repeated failures, and `ownerId`-scoped Firestore Security Rules

8. Visual Models

8.1. Flow Chart Diagram

```

flowchart TD
    Start(["App Start"]) --> Splash["Splash Screen"]
    Splash --> AuthCheck{"Logged In?"}

    AuthCheck -- No --> LoginScreen["Login Screen"]
    LoginScreen -- New user --> RegisterScreen["Register Screen"]
    RegisterScreen --> LoginScreen
    LoginScreen -- Sign-in success --> LockCheck{"App Lock Enabled?"}
    AuthCheck -- Yes --> LockCheck

    LockCheck -- Yes --> AppLockScreen["App Lock Screen  
PIN / Biometric"]
    AppLockScreen --> PinOk{"PIN / Biometric  
Correct?"}
    PinOk -- No --> Lockout["Record Failed Attempt  
Escalating Lockout"]
    Lockout --> AppLockScreen
    PinOk -- Yes --> Home["Home Dashboard"]
    LockCheck -- No --> Home

    Home --> AllCustomers["All Customers"]
    Home --> AddCustomer["Add Customer"]
    Home --> ReportsScreen["Collection Reports"]
    Home --> SettingsScreen["Settings"]
    Home --> Logout["Logout"] --> LoginScreen

    AddCustomer --> SaveCustomer["Save Customer  
ownerId = uid"]
    SaveCustomer --> AllCustomers

    AllCustomers --> CustomerDetail["Customer Detail"]
    CustomerDetail --> AddPayment["Add Payment"]
    CustomerDetail --> SetReminder["Set Reminder"]
    CustomerDetail --> CustomerPdf["Customer PDF Statement"]
    CustomerDetail --> ArchiveAction["Archive Customer"]
    ArchiveAction --> ArchivedScreen["Archived Customers"]
    ArchivedScreen -- Restore --> AllCustomers
    ArchivedScreen -- Delete --> Deleted["Removed Permanently"]

    AddPayment --> EntryType{"Payment or  
New Due?"}
    EntryType -- Payment --> Method["Choose Method  
bKash / Nagad / Cash / Bank"]
    Method --> SavePayment["Save Payment  
Update totalDue"]
    EntryType -- New Due --> SavePayment
    SavePayment --> CustomerDetail

    SetReminder --> When{"Send Now or  
Schedule?"}
    When -- Now --> SendNow["Send SMS Now"]
    SendNow --> LogSms["Log to smsLogs"]
    When -- Schedule --> Queue["Register Workmanager Task"]
    Queue --> WaitBg["Wait Until Scheduled Time  
app may be closed"]
    WaitBg --> BgSend["Background: Send SMS  
retry up to 3x"]
    BgSend --> SentOk{"SMS Sent?"}
    SentOk -- Yes --> LogSms
    SentOk -- No --> FailLog["Log Failure"]
    LogSms --> CustomerDetail

    CustomerPdf --> GeneratePdf["Generate PDF"]
    GeneratePdf --> ShareOrPrint["Share / Print"]

    ReportsScreen --> Period{"Daily / Weekly /  
Monthly?"}
    Period --> Aggregate["Aggregate Payments  
in Range"]
    Aggregate --> ExportPdf["Export Report PDF"]

    SettingsScreen --> Profile["Theme / Currency /  
Business Profile"]
    SettingsScreen --> SmsTpl["Edit SMS Template"]
    SettingsScreen --> EnableLock["Enable App Lock  
Set PIN"]
    SettingsScreen --> Backup["Export CSV Backup"]
    SettingsScreen --> RestoreCsv["Import CSV"]
    RestoreCsv --> Dedup["Skip Duplicate Phones  
Import Rest"]
    Dedup --> AllCustomers

```

Figure 1: User Flow Chart of Smart Due Manager — Mermaid source

The Mermaid source above traces the app’s user flow from launch through authentication and the app lock, into the home dashboard, and out to each major feature: customer and payment management, reminder scheduling with immediate vs. background-scheduled SMS delivery, PDF report/statement generation, and settings (theming, SMS template, app lock setup, and CSV backup/restore). Render it with any Mermaid-compatible tool (e.g. the Mermaid Live Editor at <https://mermaid.live> or the Mermaid VS Code extension) to produce the diagram image, then paste that image in place of this listing before final submission.

8.2. ERD (Entity Relationship Diagram)

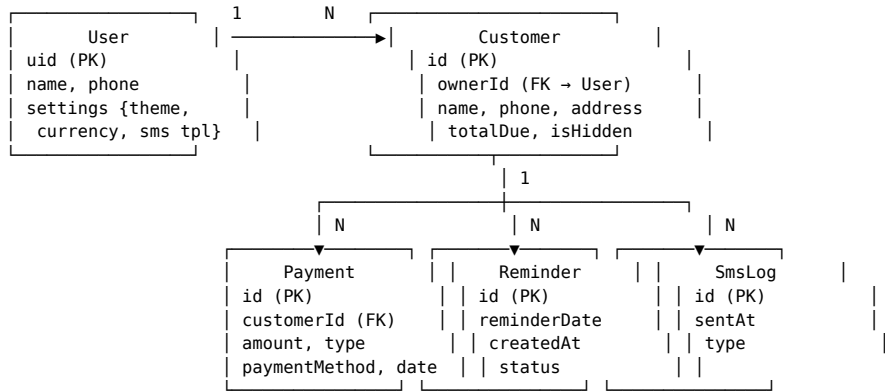


Figure 2: Entity Relationship Diagram of Smart Due Manager

The above figure illustrates the entity-relationship model backing Smart Due Manager on Cloud Firestore. Each **User** owns many **Customer** records (scoped by an `ownerId` foreign key), and each **Customer** in turn owns many **Payment**, **Reminder**, and **SmsLog** records stored as Firestore subcollections. The ERD helps in understanding the data model and how the entities relate to one another.

8.3. Timeline (Gantt Chart)

The base timeline for the development of Smart Due Manager is as follows,

Task	Wk 1-2	Wk 3-4	Wk 5-6	Wk 7-8	Wk 9	Wk 10	Wk 11	Wk 12
Requirements & UI Mockup	✓	✓						
Firebase Auth & Firestore setup		✓	✓					
Customer & payment CRUD			✓	✓				
Reminder & background SMS (Work-Manager)				✓	✓			
PDF reports & CSV backup					✓	✓		
App lock (PIN/biometric) & security rules						✓	✓	
UI polish, Bengali localization & docs							✓	✓
Final testing & deployment								✓

Table 1: Development Timeline of Smart Due Manager

The timeline is divided into 12 weeks, with specific tasks allocated to each period, describing an approximate/planned schedule for the whole development process.

8.4. UI/UX Design

Insert UI/UX wireframes or mockup screenshots here before final submission.

Figure 3: UI/UX Mockups

8.5. App screens

Insert screenshots: Login / Registration screen and Home dashboard screen.

Figure 4: Login/Register and Home Dashboard screens

Insert screenshots: Add/Detail Customer screen and Add Payment screen.

Figure 5: Customer Detail and Add Payment screens

Insert screenshots: Reminder scheduling screen and Customer PDF report screen.

Figure 6: Reminder Scheduling and PDF Report screens

Insert screenshots: Settings screen and App Lock screen.

Figure 7: Settings and App Lock screens

9. Limitations

1. Android-Only SMS Automation

Direct SMS sending is an Android-only capability; the automated reminder feature has no equivalent on iOS, where apps cannot send SMS programmatically, and no equivalent yet on desktop/web builds.

2. SIM-Dependent Delivery Cost

Because SMS is sent through the shop owner's own SIM, delivery depends on that device having network/SIM connectivity and incurs the operator's normal per-SMS cost — there is no bulk/cloud SMS gateway fallback.

3. Device-Local App Lock

The app-lock PIN protects the local app session but is device-local; it does not, by itself, encrypt data at rest beyond what Firestore/Android already provide.

4. No Analytics Dashboard

Reporting is currently limited to tabular PDF exports; there is no in-app chart/graph analytics dashboard.

5. Single Currency Setting

The project currently supports a single currency symbol setting rather than true multi-currency accounting.

10. Future Plans

1. **Cloud SMS Gateway Fallback**

Integrate an optional cloud SMS gateway as a fallback when the owner's device is offline, to complement the current on-device sending.

2. **Analytics Dashboard**

Add a visual analytics dashboard (collection trends, top overdue customers) using charts.

3. **Multi-Staff, Role-Based Accounts**

Support multiple staff accounts per business with role-based permissions, rather than one owner per dataset.

4. **Encrypted Backup & Biometric-Gated Export**

Add end-to-end encrypted backups and optional local biometric-gated data export.

5. **Expanded Automated Testing**

Extend automated test coverage to the repository and service layers using the Firestore emulator.

6. **Multi-Language Expansion**

Add further regional language options beyond Bengali and English for broader accessibility.

11. Result

Despite the limitations, the system achieves significant success in solving real-life problems of customer due and payment management:

1. **Successful Customer Due Tracking**

Owners can now add, search, and manage customer profiles and running due balances in a structured digital format.

2. **Reliable Automated SMS Reminders**

Scheduled reminders continue to fire and deliver correctly even after the app is fully closed, since the SMS-send task runs through the operating system's background scheduler.

3. **Transparent Payment History**

Every payment and due entry is preserved in an auditable history, with printable/shareable PDF statements and collection reports.

4. **Secure Multi-User Data Isolation**

Firestore Security Rules independently enforce that each user can only read or write their own customers, verified by testing with two separate accounts on the same backend.

5. **Effective App Lock Protection**

The PIN (PBKDF2-hashed) and biometric app lock, together with escalating logout, successfully prevent casual unauthorized access on a shared or lost device.

6. **Portable CSV Backup**

Customers can be exported to and re-imported from CSV, giving owners a way to move data off-device that a paper ledger cannot provide.

References

1. Khatabook. Accessed: Aug. 04, 2026. [Online]. Available: <https://khatabook.com/>
2. OkCredit. Accessed: Aug. 04, 2026. [Online]. Available: <https://okcredit.in/>
3. "Flutter." [Online]. Available: <https://flutter.dev/>
4. "Firebase Documentation." [Online]. Available: <https://firebase.google.com/docs>
5. "Cloud Firestore Security Rules Reference." [Online]. Available: <https://firebase.google.com/docs/firestore/security/get-started>
6. pub.dev, "another_telephony." [Online]. Available: https://pub.dev/packages/another_telephony
7. pub.dev, "workmanager." [Online]. Available: <https://pub.dev/packages/workmanager>

8. pub.dev, “pdf” and “printing.” [Online]. Available: <https://pub.dev/packages/pdf>, <https://pub.dev/packages/printing>
9. pub.dev, “crypto.” [Online]. Available: <https://pub.dev/packages/crypto>
10. RFC 8018, “PKCS 5: Password-Based Cryptography Specification Version 2.1 (PBKDF2),” IETF, 2017.

THE END